Assignment: Predicting and Classifying Air Pollution in Beijing

Data cleaning, EDA, feature engineering, regression, imbalance handling and classification

1. Overview

You will work with real hourly air pollution data and build two things:

1. A regression model that predicts the hourly PM2.5 value (how much fine dust is in the air).
2. A classification model that puts each hour into a health-risk level, so the agency can issue alerts.

You will go through the full workflow: cleaning, exploring, feature engineering, preprocessing, modeling, and evaluation.

2. Important Rules (Please Read First)

- Write a markdown cell above every code cell, in your own words. Explain what the cell does and why you are doing it.
- Do not use AI to write your markdown text. Any work found to be AI-written will receive zero.
- Discussion on Monday: Next Monday (28 September 2026), one student will be picked randomly to discuss their work in person. Everyone must be ready.

You must be able to explain:

  - What each plot shows and what insight you took from it.
  - Why you applied each cleaning and preprocessing step, and what would go wrong without it.
  - Why you chose each model and each evaluation metric.
- For those who will not be discussing you need to be including all of this in markdowns.

3. About the Data

Name: Beijing PM2.5 Data

Download link: https://archive.ics.uci.edu/dataset/381/beijing+pm2+5+data

Source: UCI Machine Learning Repository (dataset ID 381). Download it from the UCI website, not from Kaggle.

What it contains: About 43,800 rows. Each row is one hour of measurements in Beijing, from 2010 to 2014.

What is PM2.5? Tiny dust particles in the air, measured in µg/m³. Higher values are worse for health.

Column	Meaning
No	Row number
year, month, day, hour	Date and time of the measurement
pm2.5	PM2.5 concentration (µg/m³). This is your target.

DEWP	Dew point (°C), related to humidity
TEMP	Temperature (°C)
PRES	Air pressure (hPa)
cbwd	Combined wind direction (categories such as NE, NW, SE, cv = calm/variable)
Iws	Cumulated wind speed (m/s)
Is	Cumulated hours of snow
Ir	Cumulated hours of rain

Things to know about this data:

- Some PM2.5 values are missing, in some places as single hours and in other places as long gaps.
- The data is a time series: one hour is closely related to the hour before it.
- PM2.5 is heavily skewed: most hours are moderate, but there are a few extreme spikes.
- Is and Ir are mostly zeros.

4. Student Tasks

Part 1: Understand and Clean the Data

1. Load the data. Show its shape, column types, and summary statistics. Explain what you see.
2. Combine year, month, day, and hour into one datetime column and use it as the index. Check for missing or duplicated timestamps.
3. Find the missing values. Look at where they are: single hours, or long runs?
4. Choose a treatment for each type of gap (for example, interpolate short gaps and drop long ones). Do not fill the target with the overall mean. Explain why your choice is reasonable.
5. Look for outliers. Decide whether the high PM2.5 spikes are errors or real pollution events, and explain your reasoning.
6. Check for values that make no physical sense, or a value that repeats for many hours in a row (which could mean a broken sensor).

Part 2: Explore and Visualize

Make at least 8 plots.(more is better of course)

Under every plot, write in your own words:

- What does this plot show?
- What pattern or insight do you see?
- How will this affect what you do next?

Part 3: Feature Engineering and Preprocessing

For every step, write in your own words: what you did, why you did it, and what could go wrong if you skipped it.

Part 4: Regression

1. Train Linear Regression and SVR (try linear and RBF kernels). SVR is slow on large data, so tune it on a smaller sample of the training set and explain your choice.
2. Tune SVR's C, epsilon, and gamma using TimeSeriesSplit cross-validation.
3. Evaluate with RMSE, MAE, and R^2. If you predicted log(PM2.5), also report the results in $\mu g/m^3$.
4. Make a residual plot and a predicted-vs-actual plot. Look at how the models behave on extreme spikes.
5. Add a simple baseline: "PM2.5 equals the previous hour's value." Do your models beat it?
6. Compare the models. Is either one overfitting or underfitting? How can you tell?

Part 5: Turn the Target into Classes

Convert PM2.5 into health-risk classes using these fixed cut points (based on EPA air quality levels):

Class	PM2.5 ($\mu g/m^3$)
Good/Moderate	up to 35.4
Unhealthy for Sensitive Groups	35.5 to 55.4
Unhealthy	55.5 to 150.4
Very Unhealthy/Hazardous	150.5 and above

Then:

1. Plot how many hours fall in each class. Explain why accuracy alone can be misleading when the classes are not balanced.
2. Try at least two ways of handling imbalance, for example class_weight='balanced', SMOTE or random oversampling, and undersampling.
3. Apply resampling only to the training data, never before the split.
4. Remove PM2.5 and anything calculated from it, except lag features that use strictly past values. Explain why the lag features are acceptable.

Part 6: Classification

1. Train Logistic Regression and SVC (linear and RBF kernels) and tune their hyperparameters.
2. Evaluate with a confusion matrix, precision, recall, and F1 for each class, macro and weighted F1, and balanced accuracy.
3. Plot ROC and precision-recall curves (one-vs-rest).
4. Compare results with and without imbalance handling. Which metric for the rare classes changed, and by how much?
5. Answer in your own words: for a health alert system, is it worse to miss a hazardous hour or to raise a false alarm? Which metric matters most, and why?
6. (Optional) Make the task binary (Hazardous vs. not) and try different decision thresholds.

5. What to Submit

- A Jupyter notebook with clean, commented code and your own markdown explanation above every cell

6. Before Monday: Self-Check

You are ready for the discussion if you can answer these without looking at your notebook:

- What does each of my plots show, and what did I learn from it?
- Why did I handle missing values the way I did?
- Why did I split by time instead of randomly?
- Why did I scale the features? Which models need it?
- Why did I create each new feature?
- Why did I choose these evaluation metrics, and what does each one tell me?
- What changed after I handled the class imbalance?